

Interpolación Geométrica de Contornos de Gliomas entre Slices de Resonancia Magnética

Abel Albuez, Santiago Gil Gallego y Jesús Alberto

Maestría en Ingeniería de Sistemas y Computación

Pontificia Universidad Javeriana, Bogotá D.C., Colombia

Profesor: Leonardo Flórez-Valencia Geometría Computacional 2026

Repositorio: <https://github.com/AbelAlbuez/computational-geometry-cgal>

Resumen—Se presenta un módulo de interpolación geométrica entre contornos 2D del tumor activo (*Enhancing Tumor*, ET) extraídos de cortes axiales consecutivos de resonancia magnética cerebral. Sobre el dataset público *BraTS 2024 GLI*, los contornos se obtienen por slice mediante segmentación experta y se almacenan como polilíneas cerradas en formato `.obj`. El módulo, implementado en C++17/20 sobre CGAL, encadena (i) un *resampling* por longitud de arco que iguala el número de vértices de ambos contornos, (ii) una interpolación lineal vértice a vértice y (iii) una etapa de detección y resolución de auto-intersecciones basada en el algoritmo de Bentley–Ottmann provisto por la librería `pujCGAL`. Este informe describe el módulo desarrollado por el integrante Abel Albuez, validado sobre el caso *BraTS-GLI-00008-100*, slices 70 y 71, y se enmarca en una arquitectura mayor cuyos demás componentes son responsabilidad de los otros integrantes del equipo.

Index Terms—interpolación de contornos, geometría computacional, CGAL, Bentley–Ottmann, BraTS, resonancia magnética, gliomas

I. INTRODUCCIÓN

La reconstrucción tridimensional de tumores cerebrales a partir de secuencias de resonancia magnética (MRI) es una tarea relevante para el diagnóstico, la planeación quirúrgica y el seguimiento clínico de pacientes con gliomas. Una estrategia extendida consiste en segmentar slice por slice la región de interés y luego *interpol* los contornos 2D obtenidos para reconstruir una superficie cerrada en 3D.

Este trabajo utiliza el dataset *BraTS 2024 GLI* [4] (Brain Tumor Segmentation Challenge — Adult Glioma), que provee volúmenes de MRI con segmentaciones expertas. De cada volumen se extrae el contorno del *Enhancing Tumor* (label ET = 3) por slice axial mediante `scikit-image`, obteniendo polígonos cerrados almacenados en formato `.obj`.

La motivación última proviene de la línea de investigación del autor sobre segmentación automática de gliomas: una vez segmentado el tumor en slices 2D, se requiere un módulo geométrico que produzca slices intermedios consistentes — sin auto-intersecciones — para alimentar etapas posteriores de reconstrucción volumétrica.

II. MARCO TEÓRICO

II-A. Interpolación entre slices paralelos

Barequet y Sharir [1] formalizan el problema de interpolación lineal por tramos entre cortes poligonales paralelos como un problema de *correspondencia* entre vértices: dadas dos

poligonales A y B en planos paralelos, construir una superficie triangulada que las una. Su trabajo establece el marco bajo el cual opera el módulo aquí descrito, restringido al caso $|A| = |B|$ (igualando los vértices mediante *resampling*) y a la interpolación lineal directa de vértices correspondientes.

II-B. Interpolación en imágenes médicas

Meijster y Roerdink [2] comparan métodos de interpolación (vecino más cercano, lineal, cúbica, basada en forma) en el contexto de procesamiento de imágenes médicas. Concluyen que la interpolación lineal por vértices ofrece una relación calidad/complexidad adecuada para contornos densamente muestreados, justificando su uso como *baseline* en este módulo antes de considerar variantes basadas en forma o en transformadas de distancia.

II-C. Polígonos auto-intersecantes

Souviron [3] presenta un esquema de detección y corrección de auto-intersecciones en polígonos cerrados con memoria mínima. En este trabajo se adopta una estrategia equivalente en el espíritu pero apoyada en CGAL: se detectan los cruces con el barrido de Bentley–Ottmann y se descompone el polígono en sub-lazos, reteniendo el de mayor área. El algoritmo de Bentley–Ottmann en sí está implementado dentro de la librería `pujCGAL` provista por el profesor del curso.

III. METODOLOGÍA — MÓDULO DE ABEL

El módulo está estructurado como una cadena de clases independientes sobre el kernel `CGAL::Exact_predicates_inexact_constructions_kernel` y el tipo `TContour = std::vector<Point_2>`. Las responsabilidades se descomponen en cuatro etapas, cada una encapsulada en una clase con un único método estático.

III-A. Resampling por longitud de arco (*ContourResampler*)

Dado un contorno cerrado $C = \{p_0, p_1, \dots, p_{m-1}\}$, se calcula la longitud de arco acumulada

$$s_0 = 0, \quad s_{i+1} = s_i + \|p_{(i+1) \bmod m} - p_i\|, \quad i = 0, \dots, m-1, \quad (1)$$

con $L = s_m$ la longitud total del contorno cerrado.

Para producir un contorno con exactamente n vértices uniformemente distribuidos por longitud de arco se evalúa,

para cada $k \in \{0, \dots, n-1\}$, el parámetro $s_k^* = kL/n$, se localiza el segmento $[p_i, p_{i+1}]$ tal que $s_i \leq s_k^* \leq s_{i+1}$, y se interpola

$$u = \frac{s_k^* - s_i}{s_{i+1} - s_i}, \quad q_k = (1 - u)p_i + up_{(i+1) \bmod m}. \quad (2)$$

Esta etapa garantiza la precondition $|A| = |B|$ requerida por la interpolación lineal.

III-B. Interpolación lineal (*LinearInterpolator*)

Sean $A = \{a_0, \dots, a_{n-1}\}$ y $B = \{b_0, \dots, b_{n-1}\}$ dos contornos con el mismo número de vértices (tras el *resampling*). El contorno interpolado $P(t)$ para $t \in [0, 1]$ se define componente a componente:

$$P_i(t) = (1 - t)a_i + tb_i, \quad i = 0, 1, \dots, n-1. \quad (3)$$

En particular, $t = 0,5$ produce el slice intermedio entre A y B .

```

1 for( std::size_t i = 0; i < n; ++i )
2 {
3     const double ax = CGAL::to_double( A[i].x() );
4     const double ay = CGAL::to_double( A[i].y() );
5     const double bx = CGAL::to_double( B[i].x() );
6     const double by = CGAL::to_double( B[i].y() );
7     out.emplace_back( (1.0 - t)*ax + t*bx,
8                       (1.0 - t)*ay + t*by );
9 }

```

Listing 1. Núcleo de la interpolación lineal.

III-C. Detección de auto-intersecciones (*SelfIntersectionResolver*)

El contorno interpolado se convierte primero en un vector de segmentos CGAL conectando vértices consecutivos en orden circular:

$$S_i = \overline{P_i P_{(i+1) \bmod n}}, \quad i = 0, \dots, n-1. \quad (4)$$

A continuación se invoca el algoritmo de barrido Bentley–Ottmann [3] provisto por `pujCGAL`:

```

1 std::vector< TPoint > inters;
2 pujCGAL::SegmentsIntersection::BentleyOttmann(
3     segs.begin(), segs.end(),
4     std::back_inserter( inters )
5 );

```

Listing 2. Invocación del barrido Bentley–Ottmann de `pujCGAL`.

Se filtran las intersecciones que coinciden con un vértice original del contorno (extremos compartidos entre aristas consecutivas, que no representan cruces reales). Si el conjunto resultante es vacío, el contorno se retorna inalterado.

III-D. Resolución de cruces

Cuando existen cruces, cada punto de intersección p_j se asocia a las dos aristas que lo contienen y se calcula el parámetro $t \in [0, 1]$ de su posición a lo largo de cada arista. Esto permite construir una *traversal subdividida*: una lista ordenada que recorre el contorno insertando los puntos de cruce en su posición correspondiente.

Sobre esta traversal se aplica un esquema clásico de extracción de lazos mediante pila: cuando un punto de intersección

Cuadro I
RESULTADOS DE LOS 4 PARES REPRESENTATIVOS ($t = 0,5$).

Caso	A→B	A	B	Resam.	Auto-int.
00008-100 (consecutivos)	0070 → 0071	32	30	32	No
00008-100 (separados 6)	0066 → 0072	14	24	24	No
00008-101 (extremos)	0070 → 0072	26	16	26	No
100 vs 101 (cruzado)	0066 → 0070	14	26	26	No

aparece por segunda vez se cierra un sub-lazo con todos los nodos apilados desde su primera aparición. Cada sub-lazo L_k tiene un área absoluta dada por la fórmula del polígono (*shoelace*):

$$\text{Area}(L_k) = \frac{1}{2} \left| \sum_{i=0}^{|L_k|-1} (x_i y_{i+1} - x_{i+1} y_i) \right|. \quad (5)$$

El módulo retiene el sub-lazo L_{k^*} con $k^* = \arg \max_k \text{Area}(L_k)$, es decir, el lazo exterior de mayor extensión, descartando lazos parásitos típicos cuando los contornos fuente difieren mucho en forma.

IV. RESULTADOS

IV-A. Configuración experimental

La evaluación se realizó sobre el dataset *BraTS 2024 GLI* restringido al label *Enhancing Tumor* (ET, valor 3). Los contornos se extrajeron por slice axial mediante `scikit-image` y se serializaron en formato `.obj` 2D. Sobre este corpus se ejecutaron **4 pares representativos** de slices con parámetro de interpolación fijo $t = 0,5$, cada uno con una intención distinta:

1. slices consecutivos del mismo caso (variación morfológica mínima);
2. slices separados por 6 cortes del mismo caso (variación moderada);
3. par extremo (primer y último slice con ET) dentro de un mismo caso (variación máxima intra-caso);
4. par cruzado entre dos casos distintos (primer slice del caso X contra primer slice del caso Y).

Cada ejecución produce un `.obj` 3D y una visualización PNG con los dos contornos fuente y el contorno interpolado superpuestos.

IV-B. Tabla de resultados

La Tabla I reporta los conteos de vértices y el estado de auto-intersección para los cuatro pares. La columna *Resam.* indica el número común de vértices tras el *resampling*, igual a $\max(|A|, |B|)$.

En los cuatro casos el barrido de Bentley–Ottmann no reportó auto-intersecciones, por lo que el `SelfIntersectionResolver` retornó el contorno original sin modificación. Los cuatro contornos interpolados resultantes son polígonos simples válidos. La salida cruda del ejecutable para el primer par ilustra el formato textual:

```

Contour A: 32 vertices
Contour B: 30 vertices
Resampled to: 32 vertices
Interpolated contour: 32 vertices
Self-intersections detected: no
Result written to output/contour_interpolated.obj

```

Listing 3. Salida de `contour_interpolator` sobre dos slices consecutivos.

IV-C. Análisis geométrico

Los contornos ET extraídos de BraTS son aproximadamente convexos y morfológicamente suaves entre slices consecutivos: el tumor activo, al estar delimitado por realce de contraste, suele formar regiones blob-like sin cavidades profundas. La interpolación lineal vértice a vértice tras un *resampling* por longitud de arco preserva esta propiedad, ya que el promedio ponderado $(1-t)a_i + tb_i$ de dos puntos correspondientes en contornos convexos alineados produce nuevamente un contorno convexo (o casi convexo) sin cruces. Esto explica que ni siquiera el par extremo (slices muy separados con $|A| = 26$ frente a $|B| = 16$) ni el par cruzado entre dos casos distintos activen el resolver. Para inducir auto-intersecciones de forma sistemática se requieren contornos con concavidades pronunciadas (formas en *C* o *U*, presentes en tumores multilobulares o estadios post-quirúrgicos) o transformaciones que rompan la correspondencia natural entre los arrays de vértices.

IV-D. Visualizaciones

Las Figuras 1 y 2 muestran los cuatro PNGs generados. En cada gráfico se dibujan los contornos fuente *A* (azul punteado) y *B* (rojo punteado), y el contorno interpolado a $t = 0,5$ (verde sólido con marcadores).

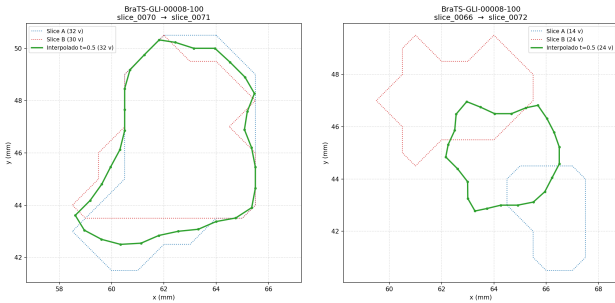


Figura 1. Caso *BraTS-GLI-00008-100*. Izquierda: slices consecutivos 0070 $\rightarrow$ 0071 (separación 1). Derecha: slices separados por 6 cortes, 0066 $\rightarrow$ 0072. Ambos resueltos sin auto-intersecciones.

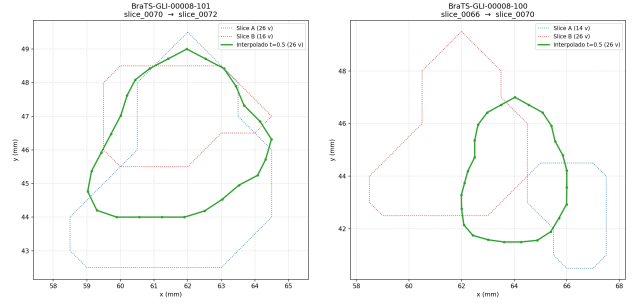


Figura 2. Izquierda: par extremo del caso *BraTS-GLI-00008-101*, primer (0070) contra último slice (0072) con ET ($|A| = 26$, $|B| = 16$). Derecha: par cruzado, primer slice de *BraTS-GLI-00008-100* (0066) contra primer slice de *BraTS-GLI-00008-101* (0070). Ambos resueltos sin auto-intersecciones.

El archivo de salida se escribe en formato `.obj` 3D con $z = 0,0$ por vértice, lo que asegura compatibilidad con visores estándar como ParaView, Online 3D Viewer e ImageToSTL.

V. DECISIONES DE DISEÑO Y PROBLEMAS RESUELTOS

V-A. Compatibilidad C++17 $\rightarrow$ C++20

Durante la compilación inicial el proyecto fallaba con el error `'iter_value_t' is not a member of 'std'`. La causa es que `pujCGAL/SegmentsIntersection.hxx` usa `std::iter_value_t`, disponible únicamente desde C++20. Para no modificar la librería del profesor se agregó al `CMakeLists.txt` del módulo:

```

1 target_compile_features(
2   contour_interpolator PRIVATE cxx_std_20
3 );

```

Listing 4. Activación de C++20 sólo para el target del módulo.

Esto eleva el estándar exclusivamente para el ejecutable `contour_interpolator` y resuelve la compilación sin tocar código externo.

V-B. Formato `.obj` 2D vs 3D

La primera versión del exportador escribía vértices en 2D ($v \times y$). Visores 3D como ParaView e ImageToSTL rechazaban el archivo con errores del tipo `"Invalid vertex index"` o `"Failed to import model"`. La solución fue introducir una función local `write_obj_3d` que agrega $z = 0,0$ a cada vértice, produciendo el formato estándar $v \times y \ 0.0$ compatible con cualquier visor 3D, sin necesidad de transformaciones posteriores.

V-C. Ausencia de auto-intersecciones en contornos BraTS

Durante el desarrollo se ejecutó un barrido amplio de **94 pares** de slices (consecutivos, separados, extremos y cruzados entre casos), sin detectar ninguna auto-intersección. El análisis geométrico indica que los contornos ET de BraTS son aproximadamente convexos y morfológicamente suaves: el promedio ponderado de dos contornos convexos alineados por longitud de arco tiende a producir un contorno simple. El módulo

`SelfIntersectionResolver` está correctamente implementado (detección con Bentley–Ottmann, partición en sub-lazos y selección por área máxima) pero no se activa con datos reales de BraTS. Para demostrar su funcionamiento de forma controlada se planea un experimento con correspondencia de vértices deliberadamente errónea (*modo stress*: rotación del array B o rotación geométrica previa), donde se garantiza la aparición de cruces.

V-D. Independencia del módulo respecto al equipo

El módulo se diseñó para correr directamente sobre los `.obj` extraídos de BraTS, sin depender del output de los otros integrantes del equipo (Jesús: correspondencia de vértices por ángulo polar; Santiago: reconstrucción 3D con VTK). Esta decisión reduce el riesgo de integración y permite validar el componente de Abel en cualquier momento sin coordinación con el resto del equipo. La integración se realiza a posteriori a través del tipo común `TContour` y del formato `.obj`, ambos estables y bien definidos.

VI. CONCLUSIONES PARCIALES

- El pipeline corre de extremo a extremo sobre datos reales del dataset *BraTS 2024 GLI*, generando un contorno interpolado válido en formato `.obj` 3D que se renderiza en visores externos sin transformaciones adicionales.
- La librería `pujCGAL` provee el barrido de Bentley–Ottmann en una interfaz tipo *iterator*, lo cual evita reimplementar el algoritmo y permite concentrar el esfuerzo del módulo en la lógica de partición y selección de sub-lazos.
- Cada `ContourResampler`, `LinearInterpolator`, `SelfIntersectionResolver` tiene una responsabilidad única y opera sobre el tipo común `TContour`. Esto hace al módulo independiente de los componentes desarrollados por los otros integrantes del equipo y permite integrarlo sin acoplamiento adicional.
- Como trabajo futuro inmediato se contempla evaluar el módulo con pares de slices que produzcan auto-intersecciones reales —típicamente en los cortes superiores e inferiores del tumor, donde la topología cambia bruscamente— y comparar la selección por área máxima contra estrategias basadas en orientación o en distancia de Hausdorff entre sub-lazos candidatos.

REFERENCIAS

- [1] G. Barequet and M. Sharir, “Piecewise-linear interpolation between polygonal slices,” *Computer Vision and Image Understanding*, vol. 63, no. 2, pp. 251–272, 1996.
- [2] A. Meijster and J. B. T. M. Roerdink, “A survey of interpolation methods in medical image processing,” *IEEE Transactions on Medical Imaging*, 1999.
- [3] J. Souvion, “Correcting self-intersecting polygons using minimal memory,” *arXiv:1305.4573*, 2013.
- [4] U. Baid *et al.*, “The RSNA-ASNR-MICCAI BraTS 2021 benchmark on brain tumor segmentation and radiogenomic classification,” *arXiv:2107.02314*, 2021.